esiea

ANGULAR

Les routes

Ressources :

https://angular.io/docs

Principe

esiea

- Angular générant des SPA (Single Page Application), la page affichée ne change pas (du moins dans son intégralité).
- Dès lors, comment gérer l'affichage des différents composants selon les besoins applicatifs ?
- C'est le principe des routes en Angular : associer une URL à une arborescence de composants inclus les uns dans les autres.
- Les routes étant communes à toute l'application, on les définit dans un fichier à la racine du projet (**`/src/app`**) dans un fichier nommé **`app.routes.ts`**.

Définition des routes

esiea

- Le contenu du fichier **`app.routes.ts`** consiste en un tableau constant (les routes sont immuables tant que l'application Angular tourne) dont le formalisme est le suivant :

```
import { Routes } from '@angular/router';
import { CardDetail } from './components/card/card-detail/card-detail';

export const routes: Routes = [
    {path:"card", component: CardDetail}
];
```

Chaque route associe une URL…

À un composant.

- Lors du développement, la route ainsi définie va donc associer l'url « **`http://localhost:4200/card`** » au composant **`CardDetail`**.

Utilisation des routes

esiea

- L'étape suivante consiste à spécifier à Angular où, dans la page afficher le composant correspondant à une route définie.

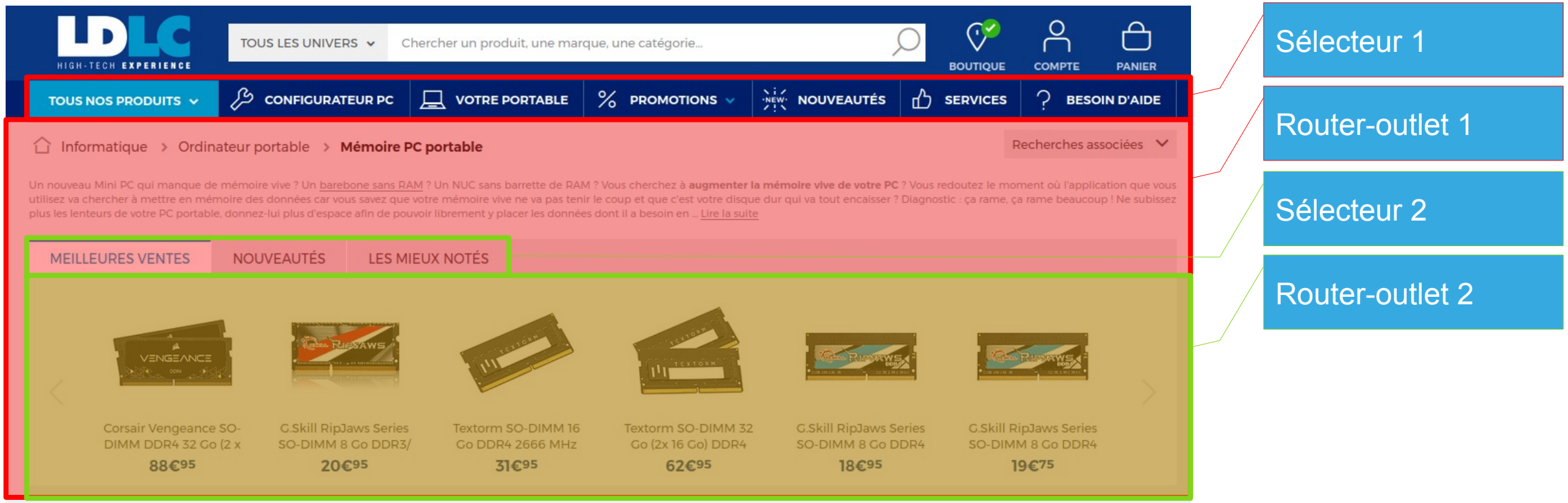
- C'est le rôle de la balise **`<router-outlet>`**.

- Il peut y avoir plusieurs balises **`<router-outlet>`** dans l'application (dans le cas de routes intriquées), mais il ne peut y en avoir qu'une dans un composant donné.

Exemple d'utilisation

esiea

- Voici un exemple de routing imbriqué basé sur la page LDLC :



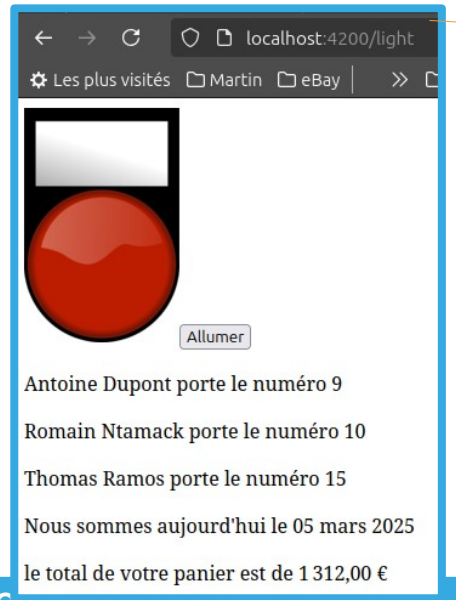
Test « à la main »

esiea

- Pour tester le routing, on peut par exemple définir les routes suivantes :

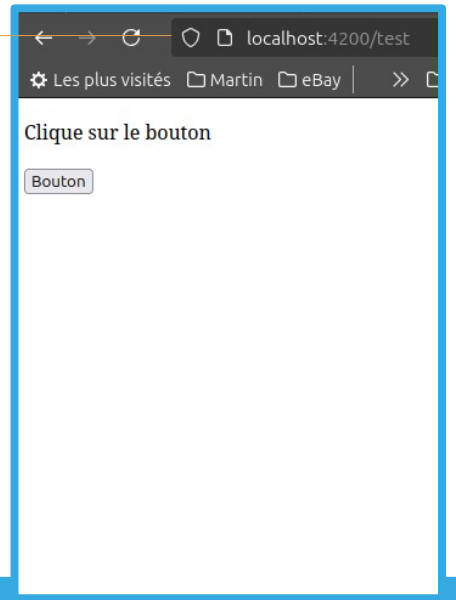
```
export const routes: Routes = [
    {path:"test", component: Test},
    {path:"light", component: Light}
];
```

- En remplaçant le code de l'**`app.html`** par une seule balise **`<router-outlet/>`**, on peut tester en changeant l'URL dans la barre d'adresse :



http://localhost:4200/light

http://localhost:4200/test



Utilisation des routes

esiea

- Il est nécessaire de créer des liens ou boutons pour permettre à l'utilisateur de naviguer vers les routes que nous avons créées.

- On pourrait penser qu'il suffit d'indiquer l'URL de nos routes dans un attribut **`href`** pour que cela fonctionne.

- C'est vrai, mais cela casserait le concept de la SPA (Single Page Application) en rechargeant systématiquement toute la page lors des appels aux serveur.

Utilisation des routes

esiea

- Pour régler ce problème, Angular nous fournit l'attribut **`routerLink`**, qui agit comme **`href`**, mais en respectant le SPA. C'est à dire qu'un clic sur le bouton ne va recharger que le contenu de la balise **`<router-outlet>`**, mais pas toute la page.

- On va donc modifier notre **`app.html`** ainsi :

```
<button routerLink="light">light</button>
<button routerLink="test">test</button>
<router-outlet></router-outlet>
```

- Attention, pour que l'attribut **`routerLink`** fonctionne dans le **`app.html`**, il faut ajouter l'import **`RouterLink`** dans le **`app.ts`** :

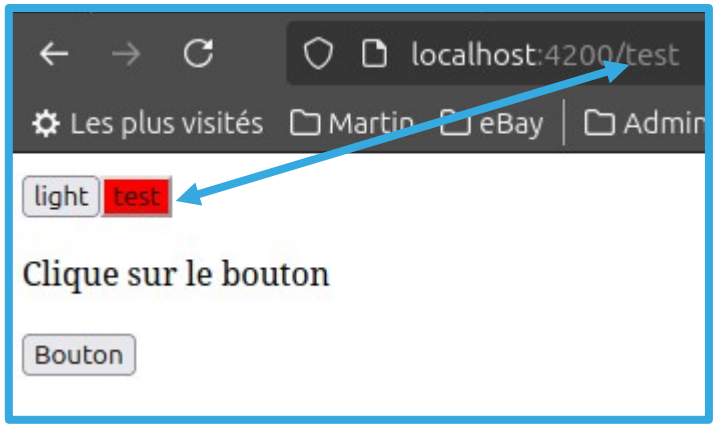
Attribut routerLinkActive

esiea

- L'attribut **`routerLinkActive`** peut être utilisé sur les balises ayant aussi l'attribut **`routerLink`** pour appliquer une classe CSS spécifique si le lien pointé est celui actif.

```
<button routerLink="light" routerLinkActive="activeLink">light</button>
<button routerLink="test" routerLinkActive="activeLink ">test</button>
```

- Comme pour l'attribut **`routerLink`**, il faut ajouter l'import **`RouterLinkActive`** dans le **`app.ts`** :
- Exemple avec un css (moche) :



Routing via le code

esiea

- Les routes peuvent êtres gérées en HTML via l'attribut **`routerLink`** mais on peut aussi avoir besoin de passer d'une route à l'autre via le code typescript.
- Pour cela, on commence par injecter le routeur dans la classe dans laquelle on veut l'utiliser.

```
constructor(private router : Router) {
}
```

- On peut ainsi utiliser ce routeur, et notamment sa méthode **`navigateByUrl(…)`** pour modifier le contenu d'une portion de page.

```
onBtnClick() : void {
  this.router.navigateByUrl("card");
}
```

Routes paramétrées

esiea

- Dans une IHM avec le pattern liste→détail, on va utiliser un composant pour afficher le détail d'un élément particulier en utilisant, par exemple, son id.

- Pour cela, on paramètre la route (dans **`app-routes.ts`**).

```
export const routes: Routes = [
    {path:"test", component: TestComponent},
    {path:"light", component: LightComponent},
    {path:"card/:id", component: CardDetailComponent}
];
```

Le paramètre de la route est préfixé par le caractère « : »

- On peut dès lors accéder au component spécifié par une URL du type : `http://localhost:4200/card/14`
où « 14 » est l'identifiant de la carte que l'on veut voir affichée dans le component card-detail.

Routes paramétrées

esiea

- Pour utiliser ce paramétrage, nous allons commencer par créer, dans le service, une méthode renvoyant une card à partir de son id (ou **`null`**, si elle n'existe pas) :

```
getCardById(id : number) :Card|null {
  for (let curCard of this.cards) {
    if (curCard.id === id) {
      return curCard;
    }
  }
  return null;
}
```

On peut renvoyer **`null`**

Attention à la différence entre **`for (… in …)`** et **`for (… of …)`**

- Jusqu'à maintenant, notre composant **`card-detail`** récupérait la fiche à afficher via la directive **`@Input`**.
- Nous allons changer cela en récupérant plutôt l'**`id`** passé via la route. Première étape, injecter le service et la route active via le constructeur :

```
constructor(private service : CardsService, private route : ActivatedRoute) {//...}
```

Routes paramétrées

esiea

- Contenu du constructeur du composant **`card-detail`** :

```
constructor(private service: CardsService, private route: ActivatedRoute) {
    const cardId: number = +this.route.snapshot.params['id'];
    let ret: Card | null = this.service.getCardById(cardId);
    if (ret !== null) {
        this.card = ret;
    }
}
```

Le « + » est une « astuce » javascript pour forcer la conversion en **`number`**.

- On peut maintenant tester cette route « à la main » en entrant dans la barre d'adresse du navigateur des URL comme **`http://localhost:4200/card/1`** en changeant l'id pour vérifier que la carte affichée change.

- Pour tester plus efficacement, créons une vraie architecture liste/détail.

Architecture liste/détail

esiea

- Une vraie architecture liste/détail, en angular est composée d'un **`router-outlet`** qui peut contenir :
  - Soit la liste qui est un composant liste contenant un **`@for`** qui affiche des composants item de liste
  - Soit un composant détail affiché dans le **`router-outlet`** quand l'utilisateur clique sur l'un des items de la liste.
- Créons les composants qui nous manquent :

```
ng g c components/card/cards-list
```

```
ng g c components/card/card-item
```

Architecture liste/détail

esiea

- Notre **`app.html`** ne contient maintenant plus que la balise **`<router-outlet>`**.
- Le composant **`cards-list`** est composé ainsi :

```
@for (card of cards; track card.id) {
    <app-card-item [card]="card"></app-card-item>
}
```

- Comme suggéré par le property binding, le composant item de liste fonctionne grâce à l'injection de la carte :

```
export class CardItem {
   @Input()
   card! : Card;
}
```

Architecture liste/détail

esiea

- Pour accéder au détail, on passe soit par l'attribut **`routerLink`** :

```
<div style="border: solid 2px #36a9e1; display: block;">
    {{card.lastName}} {{card.firsName}}
    <img [src]="card.imgUrl" style="max-height: 100px;"alt="">
    <button routerLink="card/{{card.id}}">détail</button>
</div>
```

- Soit par une méthode :

```
<div style="border: solid 2px #36a9e1; display: block;">
    {{card.lastName}} {{card.firsName}}
    <img [src]="card.imgUrl" style="max-height: 100px;"alt="">
    <button (click)="afficherDetail(card.id)">détail</button>
</div> ;
```

```
constructor(private router : Router) {}

afficherDetail(cardId : number) {
    let target : string = "/card/" + cardId;
    this.router.navigateByUrl(target);
}
```

Architecture liste/détail

esiea

- Les dernières étapes sont de prévoir un bouton de retour à la liste dans le composant de détail :

```
<div style="border: solid 2px #36a9e1;">
   <p>Nom : {{card.lastName}}</p>
   <p>Prénom : {{card.firsName}}</p>
   <p>Année de naissance : {{card.birthYear}}</p>
   <img [src]="card.imgUrl" style="max-width: 200px;"alt="">
   <button routerLink="">Retour à la liste</button>
</div>
```

- Enfin, il faut spécifier que la route par défaut doit afficher la liste (à adapter dans un vrai projet) :

```
export const routes: Routes = [
    {path:"", component: CardsListComponent},
    {path:"test", component: TestComponent},
    {path:"light", component: LightComponent},
    {path:"card/:id", component: CardDetailComponent}
];
```

Routes inconnues

esiea

- Comme pour un site web standard, il faut prendre en compte le risque d'une URL invalide et du 404 qui va suivre.

- On créé donc un composant **`page-not-found`** qui contiendra la page 404 et on lui affecte toute route autre que celles spécifiées dans le fichier **`app-routes.ts`** :

```
export const routes: Routes = [
    {path:"", component: CardsListComponent},
    {path:"test", component: TestComponent},
    {path:"light", component: LightComponent},
    {path:"card/:id", component: CardDetailComponent},
    {path:"**", pathMatch:"full", component: PageNotFoundComponent }
];
```

Le path « ** » couvre toutes Les possibilités.

C'est pourquoi la route vers la page 404 doit être la dernière du fichier !

Protection des routes

esiea

- Dans certains cas, il peut être intéressant de filtrer l'accès à certaines routes (selon le profil, par exemple).
- Pour ce faire, on utilise les **guards**. Il en existe 5 types :
  - **CanActivate** : détermine si une route peut être affichée.
  - **CanActivateChild** : détermine si on peut activer une route fille de la route actuelle.
  - **CanDeactivate** : peut-on quitter la route actuelle ? Très utile pour forcer la sauvegarde si on quitte un formulaire sans sauver avant.
  - **CanMatch** : garde générique qu'on affecte à plusieurs routes pour en choisir une.

Protection des routes

esiea

- Lorsqu'on demande à Angular de générer un **`guard`**, on peut en choisir un ou plusieurs :

Option **functional=false** pour générer le **guard** en tant que classe.

```
> ng g g guards/filter-all --functional=false
? Which type of guard would you like to create? (Press <space> to select, <a> to
toggle all, <i> to invert selection, and <enter> to proceed)
❯◯ CanActivate
 ◯ CanActivateChild
 ◯ CanDeactivate
 ◯ CanMatch
```

- Selon les choix faits, la classe de garde implémentera une ou plusieurs interfaces et une ou plusieurs méthodes seront créées. Pour chaque méthode, il suffit de renvoyer **`false`** pour que la navigation soit annulée.

Protection des routes

esiea

- Voici la classe générée avec tous les **`guards`** :

```
export class FilterAllGuard implements CanActivate, CanActivateChild, CanDeactivate<unknown>, CanMatch {
  canActivate(
    route: ActivatedRouteSnapshot,
    state: RouterStateSnapshot): MaybeAsync<GuardResult> {
    return true;
  }
  canActivateChild(
    childRoute: ActivatedRouteSnapshot,
    state: RouterStateSnapshot): MaybeAsync<GuardResult> {
    return true;
  }
  canDeactivate(
    component: unknown,
    currentRoute: ActivatedRouteSnapshot,
    currentState: RouterStateSnapshot,
    nextState?: RouterStateSnapshot): MaybeAsync<GuardResult> {
    return true;
  }
  canMatch(
    route: Route,
    segments: UrlSegment[]): MaybeAsync<GuardResult> {
    return true;
  }
}
```

Protection des routes

esiea

- Même si un seul **`guard`** peut implémenter tous les types de filtrage, il est conseillé, pour des raisons de clarté, de séparer les différents gardes selon les types et les routes à filtrer.

- On utilise les **`guards`** dans le fichier **`app.routes.ts`** en affectant un ou plusieurs gardes à une route :

```
{path:"test", canActivate:[FilterAllGuard], component: TestComponent},
```

- Notez les crochets : on peut avoir plusieurs gardes du même type pour une route donnée. Dans ce cas, ils doivent tous répondre **true** sinon la navigation n’est pas possible.